

Seoul, South Korea

+82 10-6816-3046

jlee00304@gmail.com

LinkedIn · GitHub · Portfolio

Jae Yong Lee

Frontend Engineer & AI Automation Specialist

PROFILE

Frontend engineer who built and led production TMS architecture — prioritising user experience and performance across dispatch, order, and vehicle management — as primary contributor on a 3-engineer team, contributing **64%** of repository commits. Beyond the product work, founded the company's shared component library from zero — deployed across all products — and built an agentic coding pipeline using harness engineering that runs the full development cycle without human intervention, from feature request to a PR ready for human review. Accounts for **78%** of AI automation commits and brings the same first-principles rigour to both the interfaces users rely on and the systems that help build them.

TECHNICAL SKILLS

Languages & Frameworks

TypeScript

React 19

Next.js 15.5

Vanilla Extract

Tailwind CSS

State & Data

Zustand

TanStack Query

react-hook-form

Zod

AI Engineering & Tools

Claude Code

Claude API

MCP

Harness Engineering

Agentic Pipelines

Multi-agent Orchestration

Git

Docker

PROFESSIONAL EXPERIENCE

WemeetMobility B2B Logistics SaaS

Oct 2024 – Present

Frontend Engineer & AI Automation Specialist Seoul, South Korea

AI Engineering Automation Infrastructure

Mar 2026 – Present

- Built an agentic coding pipeline with Claude Code skills and multi-agent orchestration — the goal was to cut out the instruct-check-fix-recheck back-and-forth that adds 2–3 round-trips per task. Jira ticket in, merged PR out, no one in between.
- The pipeline has **5 stages**: PRD/AS-IS ingestion → Jira ticket generation → TDD-driven implementation → PR creation with AI code review → self-healing retry on failure. Developers only verify Jira tickets and check the final output.
- Designers used to manually group and link every Figma frame before the pipeline could run. Redesigned it to traverse the frame tree downward from a single entry point — now they just point it at the top-level frame.
- The main reliability problem: skills would ignore Figma specs or rewrite existing components from scratch. Solved through harness engineering — a constraint layer defining each skill's scope, information-gap behaviour, MCP tool assignments, and output contract. Took **149 prompt iterations** to stabilise.
- Drove context engineering and token optimisation across each skill — **prepare ~50%, index-codebase ~35%, review ~28%** reduction — cutting per-ticket onboarding time by over **60%** and enabling parallel multi-task execution. Accountable for **78%** of all AI automation commits.

Claude Code

Claude API

MCP

Harness Engineering

Multi-agent Orchestration

Context Engineering

Enterprise TMS Platform (roouty-pro)

Nov 2024 – Present

- Main architect on a 3-engineer frontend team — **64% of all repository commits** (~3,322 of 5,225) across dispatch, order, vehicle, tariff, and driver management.
- Wrote and maintained **CODEBASE_AUDIT.md** — a running quality audit of 1,718 TypeScript files covering convention violations, cross-feature coupling, and ESLint disables — updated every 1–2 weeks to give refactoring decisions a concrete basis.

- Single-entry, batch, and dispatch forms each had their own validation logic that had diverged over time and re-rendered the whole form on every keystroke. Switched to **react-hook-form**'s uncontrolled mode and consolidated everything into a single **Zod** schema layer.
- Before hitting 160–200 rows in the dispatch table, it was clear the full DOM mount would cause lag. Added **TanStack Virtual** — chose it because it's headless and slots over the existing table markup without a rewrite.
- Multiple users editing the same records meant stale data showing up on screen. Applied a two-tier cache strategy: bypass cache on live operational data, long TTL on reference data, with immediate invalidation of related views on save.
- The design system Drawer handled keyboard, outside-click, and content-switch close triggers through fixed internal handlers with no interception point. Built a **custom Drawer** from scratch with each trigger independently controllable, so pre-close flows like unsaved-change warnings actually work.
- During order saves, the form would flash stale server values mid-save due to ordering dependencies across dispatch, freight brokerage, and list systems. Parallelised independent operations, chained dependent updates in sequence, and scoped per-field resets to fix it.
- After an auto-optimisation run, there was no way to tell if order edits had invalidated the dispatch result — no change log, no timestamps. Built staleness detection that distinguishes cosmetic UI interactions from server-affecting changes based on sync state, with per-type warnings and auto-reset on re-run.
- **react-hook-form**'s global dirty flag and per-field dirty states go out of sync in specific edge cases — found this while implementing unsaved-change warnings. Tracked down the timing issues to get warning logic with zero false positives and zero missed alerts.
- Save has a tricky tradeoff: wait for the full downstream chain and it feels slow; update UI early and you risk showing success when something downstream fails. Solved by updating UI right after the primary save succeeds, before downstream chains fire.
- Built the **Deck.GL** dispatch map layer — real-time polyline rendering across multiple routes and marker interactions. Was also handling all map and marker work in the platform's early development phase.

TypeScript

React 19

Next.js 15.5

Vanilla Extract

Zustand

TanStack Query

TanStack Virtual

react-hook-form

Zod

Deck.GL

• Integrated Design System (wemeet-design-system → assembo)

Oct 2024 – May 2026

- **1,098 commits and 20+ PRs** into the founding sprint — built the shared component library from zero: navigation sidebars, search bars, data tables, modals, and dropdowns, all deployed across company products.
- Led the move from wemeet-design-system — it was over-prescribed and hard to customise — to Tailwind-based **assembo**. Designed the props API for inputs, search bars, and dropdowns that need to behave differently across products while staying spec-compliant.
- assembo runs on Tailwind; roouty-pro on Vanilla Extract. Worked out a coexistence strategy that kept shared components integrating cleanly across both projects.

TypeScript

React

Tailwind CSS

Vanilla Extract

EDUCATION

University College London (UCL)

2018 – 2023 · London, United Kingdom

BSc Computer Science

- **Academic Standing:** Upper Second Class Honours (2:1) — GPA equivalent 3.6 / 4.0. Intermission fulfilled for mandatory national service (2020–2022).
- **Core Modules:** Machine Learning, AI & Neural Computing, Software Engineering, Object-Oriented Programming, IoT & Embedded Systems.

LANGUAGES

Korean | Native **English** | Professional — IELTS Academic 8.0 / 9.0